

Практическая Работа Тема 3

Работа с оперативной памятью. Циклические программы.

Циклические программы используют при обработке данных, размещенных в виде структур, что позволяют рационально разместить обрабатываемые данные в ОЗУ, повысить эффективность программы, ее быстродействие.

Массив - это набор элементов, расположенных в смежных ячейках памяти. Начальный адрес массива называют базовым адресом, а порядковый номер элемента - индексом.

Очередь (буфер FIFO «первый приходит – первый уходит») – содержит неподвижные элементы, их порядок не изменяется, а адресацию элементов выполняют два указателя для начала очереди и свободной ячейки за последним элементом. Очередь можно организовать как кольцевой буфер, в котором используется один указатель.

Стек (буфер LIFO «последний приходит – первый уходит») – одномерный массив, в котором загрузка данных и их извлечение производится только с одного конца, называемого вершиной стека. В микроконтроллерах AVR при записи байта данных в стек вначале выполняется декремент указателя стека SP (предекремент), а затем запись. При чтении порядок другой, вначале выполняется чтение, а затем инкремент SP (пост-инкремент).

Формирование и обработка массивов осуществляется циклическими программами, содержащими следующие элементы.

1 часть - инициализация. Ввод базового адреса массива, а также констант, используемых при обработке элементов массива.

2 часть - тело цикла. Математическая обработка данных. Вычисление текущего элемента. Адрес текущего элемента в циклических программах хранится в регистре, который называют указателем (Pointer). В микроконтроллерах AVR можно использовать 3 двухбайтных указателя X, Y, Z.

3 часть - условный переход выхода из цикла.

Команды условных переходов (или ветвления) имеют мнемоники, начинающиеся с букв «br» (от Branch - ветвление). Выбор направления продолжения программы происходит в зависимости от признака результата. Наиболее употребительными являются команды, анализирующие признаки C, Z, N: brcs (переход при C=1) и brcc (переход, если C=0); breq (переход, если результат равен нулю, а Z=1) и brne (переход, если результат не равен нулю); brpl (переход, если результат положительный, на старший бит равен нулю), и brmi (переход, если результат отрицательный). В командах ветвления адрес перехода обозначается меткой. Порядок выполнения процедур в программах с ветвлением определяется посредством команд условных и безусловных переходов (Рисунок 3.1).

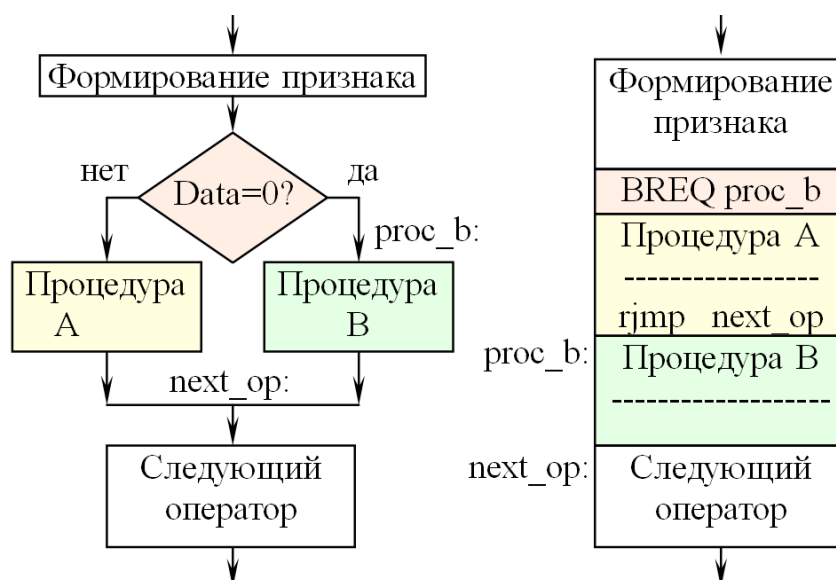


Рисунок 3.1 – алгоритм выполнения условных переходов

Команде условного перехода предшествует команда, формирующая определенный признак результата, однозначно характеризующий условие продолжения циклической обработки.

Для формирования условия окончания цикла после обработки последнего элемента массива могут использоваться различные способы. В теле цикла можно установить счетчик, анализ кода которого позволит выполнить выход из цикла. Другой вариант выхода из цикла - анализ кода указателя, и формирование по результату признака выхода из цикла.

Задание 3.1. Изучите подпрограмму pp1, (см. таблицу 3.1) которая записывает в ОЗУ три массива, каждый из которых содержит 16 элементов. Правила формирования массивов приведены в таблице 3.2.

Таблица 3.1 – пример для анализа

<pre> ; Example 4 .include "m328pdef.inc";1 ldi r16, 0x08 ;2 out sph, r16 ;3 ldi r16, 0xff ;4 out spl, r16 ;5 call pp1 ;6 ; call pp2 ;7 ; call pp3 ;8 m: jmp m ;9 pp1: ldi xh, 1 ;10 ldi yh, 1; ;11 ldi zh, 1; ;12 ldi xl, 0x00 ;13 </pre>	<pre> ldi yl, 0x10 ;14 ldi zl, 0x20 ;15 ldi r17, 10 ;16 ldi r18, 20 ;17 m1: st x+, r17 ;18 st y+, r18 ;19 mov r19, r17 ;20 add r19, r18 ;21 st z+, r19 ;22 subi r17, -3 ;23 subi r18, -5 ;24 cpi xl, 0x10 ;25 brne m1 ;26 ret ;27 </pre>	<pre> pp2: ldi xh, 0x01 ;28 clr xl ;29 ldi r16, 1 ;30 ldi r17, 0 ;31 m2: st x+, r16 ;32 add r17, r16 ;33 brcs m3 ;34 st x+, r17 ;35 add r16, r17 ;36 brcs m3 ;37 rjmp m2 ;38 m3: ret ;39 pp3: ;40 ret ;41 </pre>
--	--	--

Таблица 3.2 – правила формирования массивов

№	Базовый адрес	Выражение
1	0x0100	$X_k = 10 + 3 \cdot k$
2	0x0110	$Y_k = 20 + 5 \cdot k$
3	0x0120	$Z_k = X_k + Y_k$

Для массивов используем в качестве указателей адреса регистровые пары X, Y, Z. В строках 10 – 15 выполнена инициализация указателей – запись в регистры базовых адресов.

В регистрах r17, r18 вводятся первые элементы массивов 1 и 2 (строки 16, 17).

В теле цикла текущие значения элементов массивов записываются в ячейки памяти, адресуемые указателями (строки 18 - 22), вычисляется новые значения элемента (строки 23, 24). Заметим, что для прибавления констант 3 и 5 используются команды вычитания чисел минус 3 и минус 5.

Проверка условия выхода из цикла выполняется в конце тела цикла после инкремента указателя. Условием выхода из цикла является получение в указателе x1 адреса 0x10, который выходит за границу заданного диапазона адресов. Команда `srli` «сравнение с константой» в строке 25 выполняет неразрушающее вычитание и формирует признаки результата. В данном примере используется признак переноса C.

Введите приведенную программу, выполните отладку. Для анализа результатов отладки откройте окно View/Memory, закладка - память данных - Data.

При повторении моделирования отладчик Microchip/Atmel/AVR Studio сохраняет данные памяти, полученные ранее. Для очистки памяти необходимо установить курсор в начало таблицы данных (дампа памяти), а затем нажать и удерживать клавишу «f11».

Задание 3.2. Исследуйте работу подпрограммы `pr2` для формирования в ОЗУ последовательности чисел Фибоначчи: первые два числа равны 1, а каждое последующее равно сумме двух предыдущих. Условие окончания счета - превышение числом максимального значения для однобайтного числа. Начальный адрес массива чисел - 0x100.

В строках 28, 19 выполнена инициализация указателя X, он будет содержать адрес текущей ячейки массива.

В строках 30, 31 в массив и в аккумуляторы (r16, r17) заносятся константы для получения первых двух чисел заданной последовательности.

Дальнейшее вычисление чисел выполняется в цикле (строки 32 - 38), тело которого содержит определение двух чисел, а затем процесс повторяется, что поясняет граф. После каждого суммирования проверяется условие выхода из цикла - переполнение и появление признака переноса C=1.

Данный пример иллюстрирует вычислительные возможности процессора с блоком РОН: в теле цикла нет команд пересылки. В сложных задачах преимущества данной структуры процессора более ощутимы.

При отладке программы следует открыть окно View/Memory, закладка Data. Щелчок правой кнопкой на содержимом окна позволяет выбрать десятичную систему счисления для отображаемых данных, что упрощает анализ результата работы программы. Описание работы программы представьте в форме таблицы.

Задание 3.3. Измените программу pp2 для получения массива, содержащего 256 элементов. Попробуйте определить период псевдослучайной последовательности.

Задание 3.4. Составьте подпрограмму, определяющую максимальный элемент массива, полученного в пункте 3.3 и его запись в регистр r5.

Метод поиска максимума состоит в следующем. Вначале в регистр r5 записывается заведомо не максимальное число, например, ноль. Затем в цикле все элементы массива сравниваются с содержимым регистра r5. Если элемент массива больше содержимого регистра, то значение элемента записывается в регистр вместо предыдущего содержимого. Таким образом, в регистре формируется текущее, предполагаемое значение максимума, а в конце цикла после просмотра всего массива в регистре получим максимум.

Контрольные вопросы

1. Какие команды обеспечивают использование подпрограмм?
2. Перечислите типы арифметических операции микроконтроллеров AVR.
3. Как можно использовать арифметические операции для формирования признаков?
4. За что отвечает регистр состояния процессора? Перечислите и поясните биты регистра.
5. Опишите, изученные в работе команды условных переходов.
6. Опишите назначения указателей X, Y, Z.
7. С какого адреса начинается сегмент оперативной памяти (ОЗУ)?
8. Как описываются циклы на языке Assembler?